Université Ibn Tofail – Faculté des Sciences
Master Informatique et Intelligence Artificielle

PROCESSUS STOCHASTIQUES

Rapport de projet

Répartition intelligente de charge

Q-Learning et Double Q-Learning

Modélisation stochastique et comparaison avec les méthodes classiques

Réalisé par

M'hamed Chafi
Mouad El Haidoudi
Ilyas Yahyaoui
Oussama Benaarous
Oussama El Ouahidy

Encadré par

Pr. KAICER MOHAMMED

Année universitaire 2025–2027

Objet de l'étude. Comparer trois stratégies de répartition dans un même environnement aléatoire, puis mesurer l'apport de l'apprentissage sur le temps de réponse, les rejets et l'affectation des requêtes.

Résumé

La répartition de charge consiste à choisir le serveur qui recevra chaque nouvelle requête. Cette décision devient importante lorsque les demandes arrivent de façon irrégulière et que les serveurs ont des vitesses différentes.

Nous étudions ce problème à l'aide d'un simulateur de trois serveurs. Les arrivées, les besoins de calcul et les pannes sont aléatoires. Round Robin est comparé à deux méthodes d'apprentissage par renforcement : Q-Learning et Double Q-Learning. L'agent observe un état de charge discrétisé, choisit un serveur et reçoit une récompense qui dépend du temps de réponse estimé, de la charge et du rejet éventuel.

L'expérience repose sur cinq entraînements distincts de 5 000 épisodes. Après chaque entraînement, les trois stratégies sont testées sur les mêmes vingt graines, pour chacun des trois niveaux de trafic. Sous trafic élevé, la latence moyenne passe de **132,2 ms** avec Round Robin à **106,6 ms** avec Q-Learning et **105,2 ms** avec Double Q-Learning. Les rejets passent de **6,60 %** à environ **4,51 %**. Ces résultats montrent l'intérêt d'une décision adaptée à la charge dans l'environnement simulé étudié.

Mots-clés : répartition de charge, files d'attente, processus stochastiques, MDP, Q-Learning, Double Q-Learning.

Table des matières

1 Introduction **2**

2 État de l'art **3**

3 Modélisation stochastique du système **4**

4 Stratégies de répartition **6**

5 Protocole expérimental **7**

6 Résultats et interprétation **8**

7 Visualisation dans l'application **12**

8 Conclusion **13**

Références bibliographiques **13**

1 Introduction

Un service web peut recevoir de nombreuses requêtes en peu de temps. Pour répondre à cette demande, plusieurs serveurs travaillent ensemble. Un répartiteur de charge, ou *load balancer*, doit décider à quel serveur envoyer chaque requête.

Une première solution consiste à distribuer les requêtes à tour de rôle. Cette règle, appelée Round Robin, est simple. Cependant, donner le même nombre de requêtes à tous les serveurs n'implique pas qu'ils terminent leur travail au même moment. Un serveur lent peut accumuler une file d'attente alors qu'un serveur plus rapide dispose encore de capacité.

Le problème possède une dimension stochastique : les dates d'arrivée, les durées de traitement et la disponibilité des serveurs ne sont pas connues à l'avance. La décision doit donc tenir compte d'un système qui évolue dans l'incertitude.

1.1 Question étudiée

Un répartiteur qui apprend à partir de ses décisions peut-il réduire le temps de réponse et les rejets par rapport à une répartition à tour de rôle ?

Pour répondre à cette question, nous comparons Round Robin, Q-Learning et Double Q-Learning. Les trois méthodes utilisent les mêmes lois de trafic, les mêmes capacités de serveurs et les mêmes scénarios de test. La comparaison porte principalement sur les résultats après apprentissage.

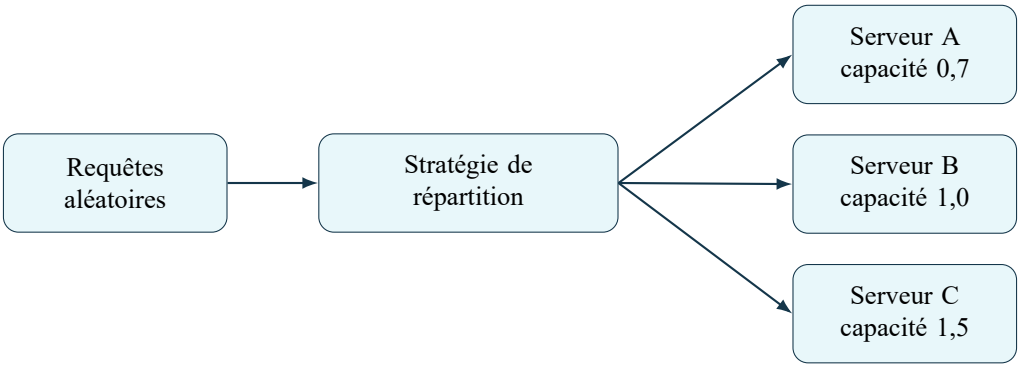


Figure 1 – Un même système de serveurs, piloté successivement par chacune des trois stratégies.

1.2 Démarche du projet

Nous présentons d'abord l'état de l'art, puis le modèle stochastique et les règles de décision. Nous décrivons ensuite un protocole reproductible et analysons les mesures obtenues. L'application permet enfin de visualiser les décisions et de relier les formules au fonctionnement du système.

2 État de l'art

2.1 Des règles classiques aux décisions adaptées à la charge

Round Robin répartit les requêtes à tour de rôle. Weighted Round Robin ajoute des poids pour tenir compte de capacités différentes. Least Connections choisit le serveur ayant le moins de connexions actives. Ces méthodes sont présentes dans des outils de répartition comme NGINX [1].

Une autre approche utilise le hasard : *Power of Two Choices* tire deux serveurs, puis retient le moins chargé. HAProxy utilise cette idée dans son algorithme aléatoire ; sa version 3.4 tient aussi compte du trafic récent lorsque les nombres de connexions sont égaux [2]. Le choix d'une stratégie dépend donc des informations disponibles sur les serveurs.

2.2 Apport des modèles stochastiques

La théorie des files d'attente relie l'intensité du trafic au temps de réponse. Dans le modèle M/M/1, avec des arrivées de Poisson de taux λ, un service exponentiel de taux μ et $\lambda < \mu$, le temps moyen dans le système vaut [3]

$$W = \frac{1}{\mu - \lambda}. \tag{1}$$

Cette formule montre que l'attente augmente fortement lorsque le trafic approche la capacité de service. Elle aide à comprendre l'enjeu de la répartition : éviter qu'un serveur soit chargé trop longtemps alors que d'autres peuvent prendre le relais.

2.3 Apprentissage par renforcement

Q-Learning apprend la valeur des décisions à partir des interactions avec un environnement [4]. Double Q-Learning sépare la sélection d'une action et l'évaluation de sa valeur future à l'aide de deux tables [5]. Cette méthode a notamment été appliquée à la répartition entre serveurs virtuels [6].

Des travaux plus récents étudient aussi des méthodes comme PPO pour adapter la répartition aux variations du trafic [7]. Ils motivent l'étude de politiques apprises, tout en rappelant que les performances doivent être mesurées dans un environnement et selon un protocole précis.

Approche	Règle de décision	Information utilisée
Round Robin	Tour de rôle	Ordre des serveurs
Weighted Round Robin	Tour de rôle pondéré	Capacités fixées
Least Connections	Moins de connexions	Connexions actives
Power of Two Choices	Meilleur de deux tirages	Mesure de charge
Q et Double Q	Action de meilleure valeur apprise	État observé et expérience

Table 1 – Principales familles de stratégies de répartition.

Positionnement. Notre étude compare une référence simple et deux méthodes tabulaires sur trois serveurs. Elle met l'accent sur la mesure des temps de réponse et des rejets lorsque la charge varie.

3 Modélisation stochastique du système

3.1 Trois serveurs et une charge de travail variable

Chaque serveur $k \in \{A, B, C\}$ possède une capacité relative c_k. Une demande de calcul D_n, exprimée en millisecondes à la vitesse de référence, nécessite un temps de service D_n/c_k sur ce serveur.

Serveur	A	B	C
Capacité relative c_k	0,7	1,0	1,5
Service pour $D_n = 50$ ms	71,4 ms	50,0 ms	33,3 ms

Table 2 – Un même travail demande des durées différentes selon le serveur choisi.

Les requêtes légères représentent 90 % des arrivées et les requêtes lourdes 10 %. Leurs demandes suivent des lois exponentielles de moyennes respectives 35 et 150 ms :

$$D_n \sim \begin{cases} \text{Exp}(1/35), & \text{avec probabilité } 0{,}9, \\ \text{Exp}(1/150), & \text{avec probabilité } 0{,}1. \end{cases} \qquad \mathbb{E}[D_n] = 46{,}5 \text{ ms}. \tag{2}$$

Le paramètre de Exp désigne ici un taux : une loi $\text{Exp}(r)$ a pour moyenne $1/r$. Ce mélange produit des requêtes de tailles différentes, dont certaines demandent beaucoup plus de travail que les autres.

3.2 Arrivées et variations du trafic

Le niveau nominal de trafic est noté u. Le simulateur utilise une durée de référence de 50 ms pour régler le taux d'arrivée. Pour la requête d'indice n, l'intervalle Δt_n est tiré selon

$$\Delta t_n \sim \text{Exp}(\lambda_n), \qquad \lambda_n = \frac{u \sum_k c_k}{50} \left[1 + 0{,}5 \sin\left(\frac{2\pi n}{70} \right) \right]. \tag{3}$$

Les taux sont exprimés par milliseconde. Le terme sinusoïdal crée des vagues de trafic au fil des requêtes. Les niveaux $u = 0{,}4$, $0{,}7$ et $0{,}9$ désignent les trois réglages de test ; ce sont des paramètres d'entrée, distincts de l'indicateur de charge de chaque serveur.

3.3 Pannes et disponibilité

À chaque arrivée, un serveur disponible peut tomber en panne avec une probabilité de 0,004. La panne dure quarante arrivées de requêtes. Pendant cette période, le serveur est exclu des choix, quelle que soit la méthode de répartition. Le simulateur permet également d'injecter une panne ou un pic de trafic pour l'observation en direct.

3.4 Files d'attente et temps de réponse

Notons B_k le travail restant sur le serveur k, en millisecondes à la vitesse de référence. Pour un serveur disponible, entre deux événements, le travail diminue avec le temps :

$$B_k(t_n^-) = \max\left(0,\ B_k(t_{n-1}^+) - c_k \Delta t_n\right). \tag{4}$$

Si la requête n est admise sur k, elle ajoute son travail à la file :

$$B_k(t_n^+) = B_k(t_n^-) + D_n, \qquad T_n = \frac{B_k(t_n^-)}{c_k} + \frac{D_n}{c_k}. \tag{5}$$

Le temps T_n est le temps de réponse estimé au moment du routage : attente puis service. Une requête est rejetée si le serveur choisi est indisponible ou si l'attente avant affectation dépasse 400 ms.

Exemple. Le serveur B possède 120 ms de travail en attente et C en possède 60 ms. Pour une nouvelle demande de 50 ms, le temps estimé vaut 170 ms sur B et $(60 + 50)/1{,}5 \simeq 73{,}3$ ms sur C. Le choix du serveur modifie donc directement l'attente de cette requête et des suivantes.

3.5 État observé et cadre de décision

Le problème est formulé dans le cadre d'un processus de décision markovien, noté $\mathcal{M} = (S, A, P, R, \gamma)$. L'agent utilise une représentation discrétisée de la charge. Pour chaque serveur, l'indicateur dérivé de la file est

$$L_k = 100 \min\left(1, \frac{B_k/c_k}{200}\right). \tag{6}$$

Il est classé en trois niveaux : faible si $L_k < 40$, moyen si $40 \le L_k \le 70$, élevé si $L_k > 70$. L'état observé $s = (\ell_A, \ell_B, \ell_C)$ comporte ainsi $3^3 = 27$ possibilités. Le simulateur fait évoluer les files et les événements ; l'agent consulte sa table à partir de ces vingt-sept observations.

Les trois actions sont les choix de A, B ou C, sous réserve de disponibilité. Les transitions P sont obtenues par simulation. Les algorithmes apprennent à partir des observations successives, sans construire une matrice complète de probabilités.

3.6 Récompense associée à une décision

La récompense utilisée est

$$r_n = -\left(\frac{T_n}{100} + 0{,}5\frac{L_k}{100} + 5F_n + O_n\right), \tag{7}$$

où F_n vaut 1 en cas de rejet et O_n vaut 1 pour une requête admise portant L_k au-delà de 90 %. Pour un rejet, le terme de temps est fixé à 500 ms. La normalisation met le temps et la charge à une échelle comparable. Avec 85 ms, 50 % de charge et aucun rejet ni surcharge, on obtient $r_n = -(0{,}85 + 0{,}25) = -1{,}10$. Maximiser cette récompense négative revient à réduire les coûts.

4 Stratégies de répartition

Une politique π associe une décision à chaque observation. L'apprentissage vise à augmenter le gain actualisé $G = \sum_{n=0}^{199} \gamma^n r_n$ d'un épisode de deux cents requêtes. Une décision est ainsi évaluée pour sa récompense immédiate et pour les situations qu'elle prépare.

4.1 Round Robin

Round Robin choisit A, puis B, puis C, et recommence. Il saute les serveurs indisponibles, mais ne consulte pas leur charge. Il fournit une référence commune, sans phase d'apprentissage.

Pour simplifier l'étude, nous retenons Round Robin comme référence classique unique. Sa règle simple, son faible coût de décision et sa facilité de reproduction permettent de concentrer la comparaison sur l'apport de l'apprentissage.

4.2 Q-Learning

Une table $Q(s, a)$ contient vingt-sept lignes et trois colonnes. Chaque case estime l'intérêt de choisir un serveur dans une situation donnée. Après une transition (s_n, a_n, r_n, s_{n+1}), la mise à jour est [4]

$$Q(s_n, a_n) \leftarrow Q(s_n, a_n) + \eta \left[r_n + \gamma \max_{a' \in A_{n+1}} Q(s_{n+1}, a') - Q(s_n, a_n) \right]. \tag{8}$$

Le taux η règle l'ampleur de la correction et γ le poids des récompenses futures. A_{n+1} contient les choix disponibles. À la fin d'un épisode, la valeur future est fixée à zéro.

Pendant l'entraînement, la règle ε-greedy choisit une action disponible au hasard avec probabilité ε ; sinon elle retient une action de valeur maximale. Les égalités sont départagées aléatoirement. Pendant le test, $\varepsilon = 0$ et les tables restent fixes.

Exemple de correction. Avec $Q = -10$, $r = -1{,}8$, $\max Q(s', \cdot) = -8$, $\eta = 0{,}1$ et $\gamma = 0{,}9$, la cible vaut $-1{,}8 + 0{,}9(-8) = -9$. La nouvelle valeur est $-10 + 0{,}1[-9 - (-10)] = -9{,}9$: la décision a été un peu meilleure que prévu.

4.3 Double Q-Learning

Double Q-Learning utilise deux tables Q_A et Q_B. À chaque interaction, une table est choisie avec probabilité $1/2$ pour être corrigée. Si c'est Q_A, on écrit [5]

$$a^* \in \arg\max_{a' \in A_{n+1}} Q_A(s_{n+1}, a'), \tag{9}$$

$$Q_A(s_n, a_n) \leftarrow Q_A(s_n, a_n) + \eta \left[r_n + \gamma Q_B(s_{n+1}, a^*) - Q_A(s_n, a_n) \right]. \tag{10}$$

Pour corriger Q_B, les rôles sont inversés. Une table sélectionne donc l'action suivante, l'autre en fournit la valeur. Pour prendre une décision, l'agent utilise la moyenne $(Q_A + Q_B)/2$.

> Si $Q_A(s', \cdot) = (8, 6, 5)$ et $Q_B(s', \cdot) = (4, 7, 3)$, Q_A sélectionne A, mais Q_B l'évalue à 4. La valeur future utilisée est donc 4. Cette séparation vise à réduire le biais lié au maximum des estimations.

Chaque simulation possède un seul décideur. Les deux algorithmes d'apprentissage sont des stratégies alternatives, comparées sur les mêmes événements.

5 Protocole expérimental

5.1 Cinq entraînements distincts

Chaque répétition repart de tables nulles. Les paramètres de l'environnement et les règles de mise à jour sont ceux de l'application. Des graines différentes sont utilisées pour les décisions aléatoires, les niveaux de trafic et les réalisations du trafic d'entraînement. Au sein d'une répétition, Q-Learning et Double Q-Learning reçoivent les mêmes arrivées, demandes et pannes.

Élément	Valeur retenue
Répétitions d'entraînement	5
Épisodes par répétition et par méthode	5 000
Requêtes par épisode	200
Décisions d'entraînement par méthode et par répétition	1 000 000
Taux d'apprentissage η et actualisation γ	0,1 et 0,9
Exploration ε	De 1 à 0,05 en 4 000 épisodes
Niveau de trafic d'un épisode	Tiré uniformément entre 0,3 et 1,0
Scénarios de test	$u = 0{,}4\,;\, 0{,}7\,;\, 0{,}9$
Graines de test par scénario	20, communes aux trois méthodes
Politique pendant le test	Tables fixes, $\varepsilon = 0$

Table 3 – Paramètres de la campagne de mesure.

Les graines maîtresses sont 101 à 105. Les plages des graines de trafic d'entraînement commencent à 10 000, 30 000, 50 000, 70 000 et 90 000, avec 5 000 épisodes par plage. Le test utilise les graines 1 000 000 à 1 000 019. Il n'y a ainsi aucun recouvrement entre trafic d'entraînement et trafic de test.

5.2 Indicateurs et calcul des moyennes

La latence moyenne et le p95 portent sur les temps de réponse estimés des requêtes admises. Le taux de rejet est le nombre de rejets immédiats divisé par le nombre d'arrivées. Le débit d'admission est le nombre de requêtes admises divisé par la durée simulée. Nous suivons aussi la part des requêtes de chaque serveur et l'écart entre les indicateurs de charge maximal et minimal.

Pour un indicateur m, nous calculons d'abord une moyenne sur les vingt graines de test pour chaque répétition r, puis une moyenne sur les cinq entraînements :

$$m_r = \frac{1}{20}\sum_{j=1}^{20} m_{r,j}, \qquad \bar{m} = \frac{1}{5}\sum_{r=1}^{5} m_r, \qquad s_m = \sqrt{\frac{1}{4}\sum_{r=1}^{5}(m_r - \bar{m})^2}. \tag{11}$$

Les tableaux donnent $\bar{m} \pm s_m$: la dispersion concerne les **cinq entraînements**, chacun évalué sur les mêmes vingt graines. Round Robin n'étant pas entraîné, sa moyenne sur ces vingt graines constitue une référence identique pour les cinq répétitions. Les p95 présentés sont des moyennes de p95 calculés par épisode.

6 Résultats et interprétation

6.1 Évolution pendant l'apprentissage

Les deux méthodes commencent avec des tables nulles et une exploration totale. Elles essaient donc de nombreuses décisions avant de privilégier les actions apprises. La figure 2 présente la récompense par épisode, lissée sur cent épisodes, ainsi que la diminution de ε.

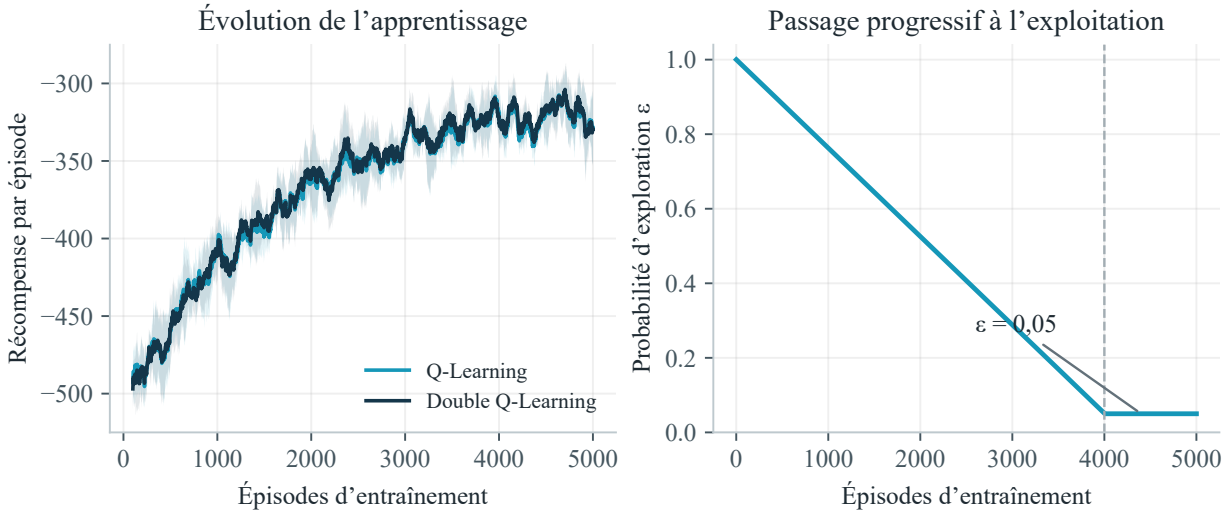


Figure 2 – À gauche, moyenne des cinq entraînements et bande d'un écart-type après lissage sur cent épisodes. Une récompense plus proche de zéro est meilleure. À droite, règle d'exploration commune aux cinq répétitions.

L'amélioration de la récompense indique que les décisions coûtent moins cher selon la fonction (7). Elle associe une réduction des temps estimés et des pénalités de charge ou de rejet. La baisse de ε donne progressivement plus de place à la politique apprise.

Le niveau de trafic change à chaque épisode. Les courbes gardent donc des fluctuations, même après de nombreux échanges avec le simulateur. Le lissage permet de suivre la tendance générale sans confondre un épisode calme avec une amélioration durable.

6.1.1 Du réglage de l'exploration à la comparaison finale

Pendant les quatre mille premiers épisodes, ε diminue linéairement de 1 à 0,05. Les mille derniers épisodes conservent cette petite part d'exploration. Lors du test, celle-ci est désactivée : les tableaux des pages suivantes mesurent les décisions des politiques finales, sans mise à jour des tables.

Deux lectures complémentaires. La courbe montre comment l'agent apprend. Le test mesure ce que sa politique permet d'obtenir une fois l'apprentissage terminé. La comparaison avec Round Robin repose sur cette seconde phase.

Le nombre de décisions est identique pour Q-Learning et Double Q-Learning : un million par répétition. Le coût supplémentaire de Double Q-Learning correspond à la gestion de deux petites tables de valeurs ; il ne modifie pas le nombre de requêtes observées.

6.2 Temps de réponse moyen et latence p95

Les deux stratégies apprises réduisent en moyenne la latence dans les trois niveaux de trafic. L'écart avec Round Robin augmente lorsque le trafic devient plus important.

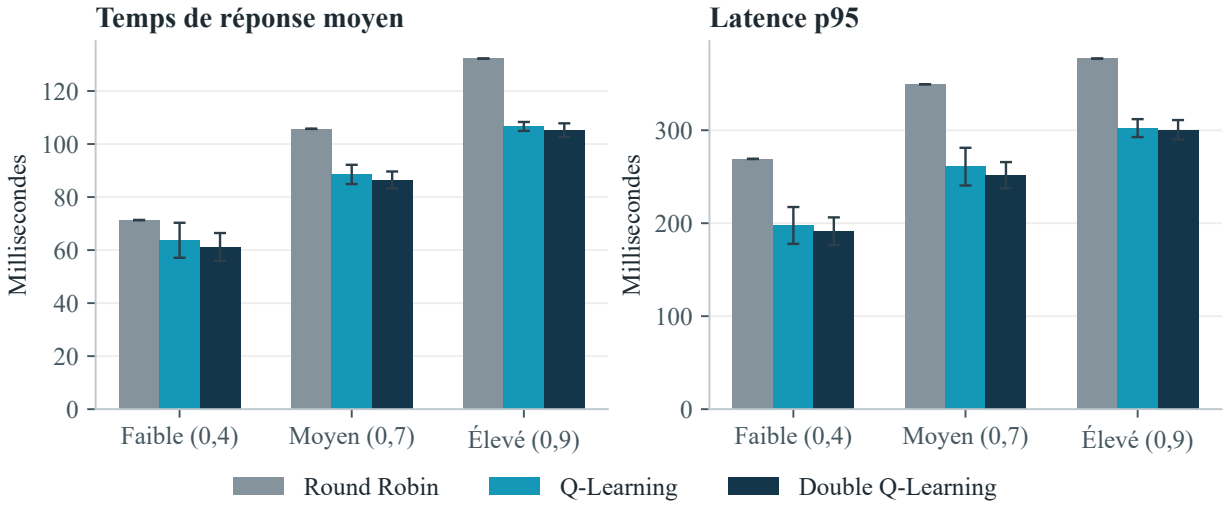


FIGURE 3 – Latence moyenne et p95 après apprentissage. Les barres d'erreur représentent la dispersion des moyennes entre les cinq entraînements, selon le protocole décrit au chapitre 5.

Trafic	Méthode	Latence moyenne (ms)	p95 (ms)
Faible	Round Robin	71,3	269,1
	Q-Learning	63,7 ± 6,6	197,6 ± 19,8
	Double Q-Learning	61,2 ± 5,2	191,4 ± 14,9
Moyen	Round Robin	105,7	349,5
	Q-Learning	88,5 ± 3,6	260,8 ± 20,3
	Double Q-Learning	86,5 ± 3,2	251,6 ± 14,1
Élevé	Round Robin	132,2	377,0
	Q-Learning	106,6 ± 1,7	302,3 ± 9,6
	Double Q-Learning	105,2 ± 2,6	300,4 ± 10,5

TABLE 4 – Temps estimés des requêtes admises. Pour les méthodes apprises : moyenne ± écart-type entre entraînements ; pour Round Robin : référence sur vingt graines.

À trafic élevé, Q-Learning réduit la latence moyenne de **19,4 %** et Double Q-Learning de **20,4 %** par rapport à Round Robin. Le p95 passe de 377,0 ms à 302,3 et 300,4 ms respectivement. L'apprentissage améliore donc aussi la partie haute des temps de réponse, au-delà du seul comportement moyen.

Double Q-Learning obtient ici une latence moyenne légèrement plus faible que Q-Learning. L'écart entre les deux est cependant nettement plus petit que leur gain commun face à Round Robin. Le résultat principal est l'intérêt de tenir compte de la charge, et non l'existence d'un avantage uniforme d'un algorithme d'apprentissage sur l'autre.

6.3 Rejets et débit d'admission

Une faible latence doit être lue avec le nombre de requêtes admises : écarter davantage de demandes pourrait rendre les temps des seules requêtes restantes trompeurs. Les résultats montrent ici une réduction simultanée de la latence et des rejets moyens.

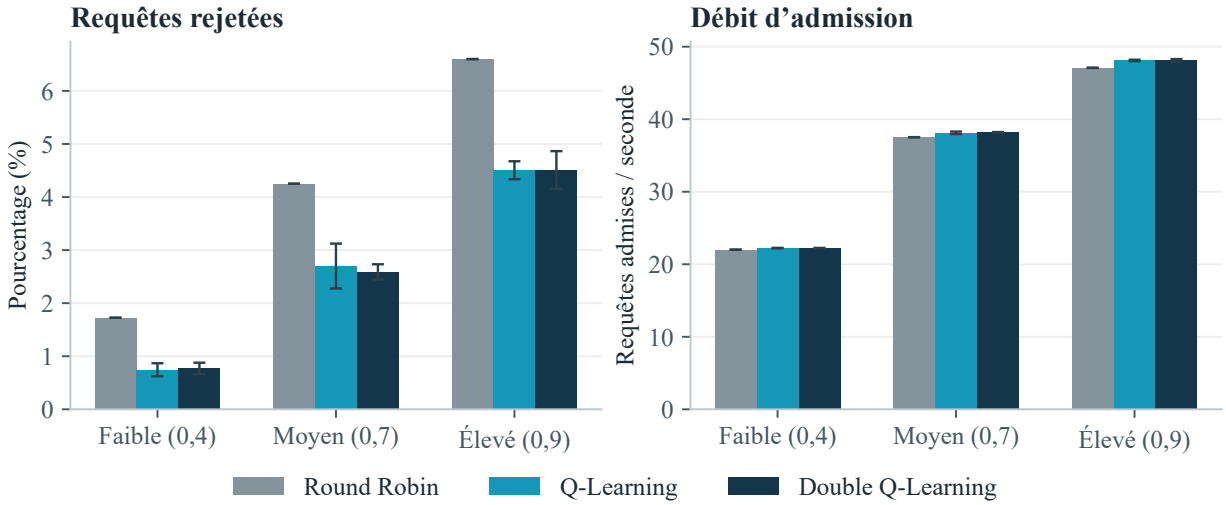


FIGURE 4 – Taux de rejet immédiat et débit d'admission. Ce débit compte les requêtes admises par seconde simulée. Les barres d'erreur portent sur les cinq entraînements.

Trafic	Méthode	Rejets (%)	Débit admis (req/s)
Faible	Round Robin	1,73	22,01
	Q-Learning	0,75 ± 0,12	22,23 ± 0,03
	Double Q-Learning	0,77 ± 0,11	22,23 ± 0,02
Moyen	Round Robin	4,25	37,53
	Q-Learning	2,70 ± 0,42	38,13 ± 0,17
	Double Q-Learning	2,59 ± 0,14	38,18 ± 0,05
Élevé	Round Robin	6,60	47,08
	Q-Learning	4,51 ± 0,17	48,11 ± 0,10
	Double Q-Learning	4,51 ± 0,36	48,10 ± 0,20

TABLE 5 – Rejets et admissions, avec la même convention de moyenne et de dispersion que pour la latence.

Sous trafic élevé, les deux méthodes apprises présentent environ 4,51 % de rejets, contre 6,60 % pour Round Robin. La baisse est de **2,09 points de pourcentage**, soit environ **32 % de rejets en moins** relativement à la référence.

Le débit d'admission augmente plus modérément, de 47,08 à environ 48,10 requêtes par seconde. Le volume d'arrivées étant commun aux méthodes, cette hausse vient des demandes supplémentaires qui sont admises. La comparaison montre ainsi un progrès conjoint : des réponses estimées plus rapides et moins de rejets à l'arrivée.

6.4 Répartition des requêtes et régularité des gains

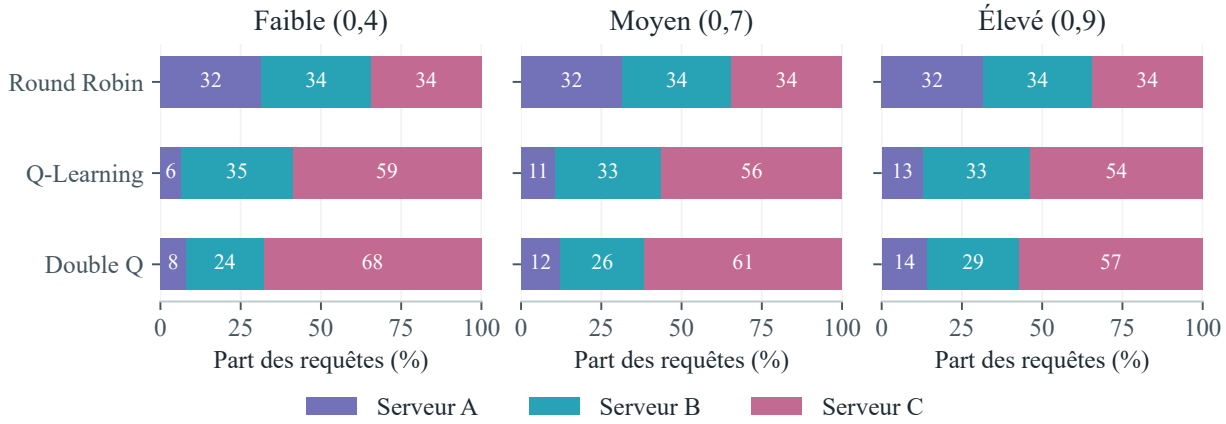


FIGURE 5 – Part moyenne des requêtes affectées à A, B et C, sur les cinq entraînements et les vingt graines de test. Les serveurs indisponibles sont exclus des choix.

Round Robin attribue des parts proches d'un tiers. Les méthodes apprises orientent davantage de requêtes vers C, dont la capacité est la plus élevée. Sous trafic élevé, C reçoit environ 54 % des affectations avec Q-Learning et 57 % avec Double Q-Learning, contre 34 % avec Round Robin. L'agent adapte sa décision à la charge observée et à l'effet qu'a eu chaque serveur sur les récompenses.

Cette répartition réduit aussi l'écart moyen entre les indicateurs de charge : à trafic élevé, il passe de 67,9 points avec Round Robin à 49,9 avec Q-Learning et 48,1 avec Double Q-Learning. Répartir le travail de façon adaptée ne signifie donc pas donner à tous le même nombre de requêtes.

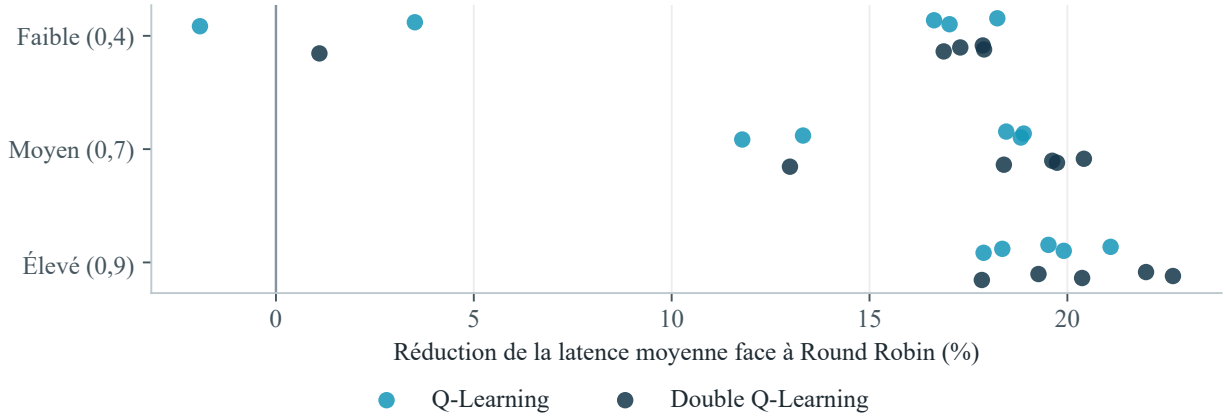


FIGURE 6 – Chaque point représente un entraînement : réduction de sa latence moyenne face à la référence Round Robin, sur les mêmes vingt graines. Une valeur positive correspond à une réduction du temps.

Aux trafics moyen et élevé, les cinq entraînements de chaque méthode obtiennent tous un gain de latence face à Round Robin. Sous trafic élevé, ce gain se situe entre 17,9 et 21,1 % pour Q-Learning, et entre 17,8 et 22,7 % pour Double Q-Learning. À faible trafic, la dispersion est plus marquée : les moyennes sont donc complétées par les points individuels.

7 Visualisation dans l'application

L'application relie le modèle aux décisions observables. Elle présente l'état des serveurs, la politique apprise, les valeurs consultées et les indicateurs. Elle permet de lancer l'entraînement, d'effectuer le test commun et de suivre une simulation en direct. Les calculs sont réalisés en Python ; une interface web communique avec eux par une API Flask.

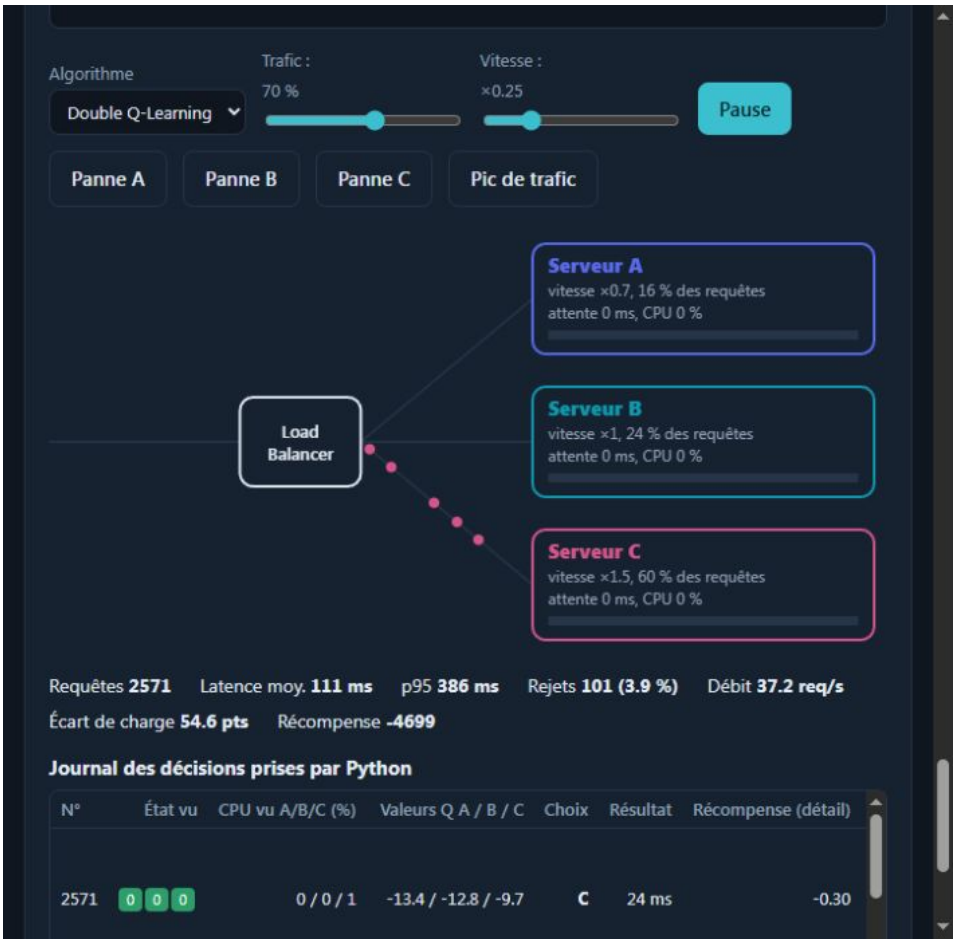


FIGURE 7 – Capture réelle d'une session de démonstration avec Double Q-Learning. Les points représentent les requêtes routées ; les blocs montrent les trois serveurs. Les mesures du chapitre 6 proviennent de la campagne de test, distincte de cette session illustrative.

Le journal associe à chaque requête l'état vu, les valeurs Q et le serveur retenu. Il permet de vérifier le lien entre $a = \arg\max Q(s, a)$ et la décision affichée. Les commandes de panne et de pic de trafic rendent visibles les changements de disponibilité et l'évolution des files. Cette vue complète les comparaisons chiffrées par une lecture concrète du mécanisme.

8 Conclusion

Ce projet étudie une décision simple en apparence : choisir le serveur qui recevra la prochaine requête. Les arrivées et les besoins de calcul étant aléatoires, une répartition à parts presque égales ne suffit pas à obtenir des temps de réponse comparables sur des serveurs de capacités différentes.

Le simulateur permet d'exprimer ce problème à l'aide de files de travail, d'un état de charge discrétisé et d'une récompense commune. Q-Learning et Double Q-Learning apprennent leurs décisions en interagissant avec cet environnement, puis sont évalués sans exploration sur les mêmes scénarios que Round Robin.

Les cinq répétitions montrent un apport net de l'apprentissage sous trafic moyen et élevé. Au niveau élevé, les temps moyens estimés diminuent d'environ **19 à 20 %**, les p95 passent d'environ **377 à 300 ms**, et le taux de rejet passe de **6,60 % à 4,51 %**. Les politiques apprises utilisent davantage le serveur rapide et réduisent l'écart de charge entre les serveurs.

Double Q-Learning obtient ici des latences moyennes légèrement plus faibles, tandis que les rejets des deux méthodes restent très proches. La comparaison met surtout en évidence l'intérêt d'adapter le routage à l'état du système et aux conséquences des décisions passées. Dans le cadre simulé étudié, cette adaptation apporte un gain mesurable face à la règle du tour de rôle.

Références

[1] NGINX. *HTTP Load Balancing*. Documentation officielle. https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/.

[2] HAProxy Technologies. *Announcing HAProxy 3.4*, 2026. https://www.haproxy.com/blog/announcing-haproxy-3-4.

[3] IIT Bombay / NPTEL. *Queuing Analysis*. Cours sur les files d'attente. https://www.civil.iitb.ac.in/tvm/nptel/585_Queuing.html.

[4] C. J. C. H. Watkins et P. Dayan. *Q-learning*. Machine Learning, 8, 279–292, 1992. doi :10.1007/BF00992698.

[5] H. van Hasselt. *Double Q-learning*. Advances in Neural Information Processing Systems, 23, 2613–2621, 2010. Article original, NeurIPS.

[6] D. Tennakoon, M. Chowdhury et T. H. Luan. *Cloud-based load balancing using double Q-learning for improved Quality of Service*. Wireless Networks, 29, 1043–1050, 2023 ; publication en ligne en 2018. doi :10.1007/s11276-018-1888-8.

[7] R. Z. Al-Shaikh, M. M. Jawad Al-Nayar et A. M. Hasan. *Reinforcement Learning algorithms for adaptive load-balancing for Web applications*. South African Computer Journal, 37(2), 121–145, 2025. doi :10.18489/sacj.v37i2.20753.